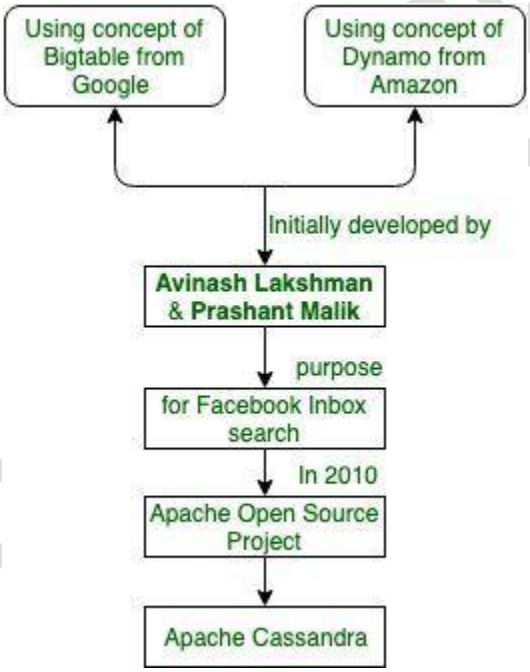
Unit-03
Introduction to Cassandra

❖ 3.1 Overview of Cassandra

Cassandra is a distributed database management system which is open source with wide column store, NoSQL database to handle large amount of data across many commodity servers which provides high availability with no single point of failure. It is written in Java and developed by Apache Software Foundation. **Avinash Lakshman** & **Prashant Malik** initially developed the Cassandra at Facebook to power the Facebook inbox search feature. Facebook released Cassandra as an open source project on Google code in July 2008. In March 2009 it became an Apache Incubator project and in February 2010 it becomes a top-level project. Due to its outstanding technical features Cassandra becomes so popular.



- ➢ Apache Cassandra is used to manage very large amounts of structure data spread out across the world. It provides highly available service with no single point of failure.
- ➢ Listed below are some points of Apache Cassandra:

- It is scalable, fault-tolerant, and consistent.
- It is column-oriented database.
- Its distributed design is based on Amazon's Dynamo and its data model on Google's Big table.

It is Created at Facebook and it differs sharply from relational database management systems.

- ➢ Cassandra implements a Dynamo-style replication model with no single point of failure but its add a more powerful "column family" data model. Cassandra is being used by some of the biggest companies such as Facebook, Twitter, Cisco, Rackspace, eBay, Netflix, and more. The design goal of a Cassandra is to handle big data workloads across multiple nodes without any single point of failure. Cassandra has peer-to-peer distributed system across its nodes, and data

is distributed among all the nodes of the cluster. All the nodes of Cassandra in a cluster play the same role. Each node is independent, at the same time interconnected to other nodes. Each node in a cluster can accept read and write requests, regardless of where the data is actually located in the cluster. When a node goes down, read/write request can be served from other nodes in the network. Features of Cassandra: Cassandra has become popular because of its technical features. There are some of the features of Cassandra:

1. **Easy data distribution –** It provides the flexibility to distribute data where you need by replicating data across multiple data centers. for example: If there are 5 node let say N1, N2, N3, N4, N5 and by using partitioning algorithm we will decide the token range and distribute data accordingly. Each node have specific token range in which data will be distribute. let's have a look on diagram for better understanding.
2. **Flexible data storage** – Cassandra accommodates all possible data formats including: structured, semi-structured, and unstructured. It can dynamically accommodate changes to your data structures accordingly to your need.
3. **Elastic scalability** – Cassandra is highly scalable and allows to add more hardware to accommodate more customers and more data as per requirement.
4. **Fast writes** – Cassandra was designed to run on cheap commodity hardware. Cassandra performs blazingly fast writes and can store hundreds of terabytes of data, without sacrificing the read efficiency.
5. **Always on Architecture** – Cassandra has no single point of failure and it is continuously available for business-critical applications that can't afford a failure.
6. **Fast linear-scale performance** – Cassandra is linearly scalable therefore it increases your throughput as you increase the number of nodes in the cluster. It maintains a quick response time.
7. **Transaction support** – Cassandra supports properties like Atomicity, Consistency, Isolation, and Durability (ACID) properties of transactions.

❖ 3.2 Data model in Cassandra

The data model of Cassandra is significantly different from what we normally see in an RDBMS. This chapter provides an overview of how Cassandra stores its data.

1. Keyspace: A keyspace is the highest-level container for data in Cassandra, analogous to a schema in a relational database. It holds tables and defines the replication strategy and other settings for the tables within it.

2. Table: Tables in Cassandra are similar to tables in relational databases but have some key differences. They consist of rows and columns, with each row identified by a primary key. Tables are distributed across the cluster according to the partitioning strategy defined by the primary key.

3. Primary Key: The primary key is a unique identifier for rows within a table. It consists of one or more columns. In Cassandra, the primary key is composed of two parts:

4. Partition Key: One or more columns that determine the partition within which a row will be stored. Data with the same partition key will be stored together on the same node.

5. Clustering Columns: Additional columns that define the clustering order within a partition. They determine how data is sorted within a partition.

6. Partitioning: Cassandra distributes data across the cluster based on the partition key. Each node in the cluster is responsible for a range of partitions, determined by a token range.

7.Replication: Cassandra replicates data across multiple nodes to ensure fault tolerance and high availability. The replication strategy determines how many copies of data are stored and where they are placed in the cluster.

8. Data Types: Cassandra supports various data types, including primitive types like text, integer, boolean, and more complex types like collections (lists, sets, and maps) and user-defined types.

9. Secondary Indexes: Cassandra supports secondary indexes, which allow querying data based on columns other than the primary key. However, the use of secondary indexes should be carefully considered due to potential performance implications.

10. Materialized Views: Cassandra provides support for materialized views, which allow you to create denormalized views of your data to optimize queries for specific access patterns.

❖ 3.3 CQL (Cassandra Query Language)

Cassandra Query Language (CQL) is the primary interface for interacting with Apache Cassandra databases. It is a SQL-like language specifically designed for working with Cassandra's distributed architecture and data model. Here are some key features and aspects of CQL:

1. **SQL-like Syntax:** CQL has a syntax that is similar to SQL (Structured Query Language), making it relatively easy for developers familiar with SQL to work with Cassandra.
2. **Keyspace Management:** CQL provides commands for creating, altering, and dropping keyspaces. Keyspaces serve as logical containers for tables and define replication strategies.
3. **Table Operations:** CQL supports commands for creating, altering, and dropping tables. Developers can define columns, primary keys, clustering columns, and other table properties using CQL.
4. **Data Manipulation:** CQL provides commands for inserting, updating, deleting, and querying data within tables. It supports basic CRUD (Create, Read, Update, Delete) operations.
5. **Primary Key Definition:** CQL allows developers to define primary keys for tables, specifying partition keys and clustering columns. This determines how data is distributed and sorted within the cluster.
6. **Secondary Indexes:** CQL supports the creation of secondary indexes, which allow querying based on columns other than the primary key. However, secondary indexes should be used judiciously due to potential performance implications.
7. **Batch Operations:** CQL supports batch operations, allowing multiple data modification queries to be executed atomically within a single batch.
8. **User-defined Types:** CQL enables the creation of user-defined types (UDTs), allowing developers to define custom data types with multiple fields.
9. **Collections:** CQL supports collections such as lists, sets, and maps, allowing developers to store multiple values within a single column.
10. **Aggregation Functions:** CQL provides aggregation functions like COUNT, SUM, AVG, MIN, and MAX for performing calculations on data within tables.

11. **Prepared Statements:** CQL supports prepared statements, which can improve performance and prevent SQL injection attacks by allowing queries to be precompiled and reused.
12. **Consistency Levels:** CQL allows developers to specify consistency levels for read and write operations, providing control over data consistency and availability guarantees.

❖ 3.4 Installing and configuring Cassandra

Apache Cassandra and Datastax enterprise is used by different organization for storing huge amount of data.

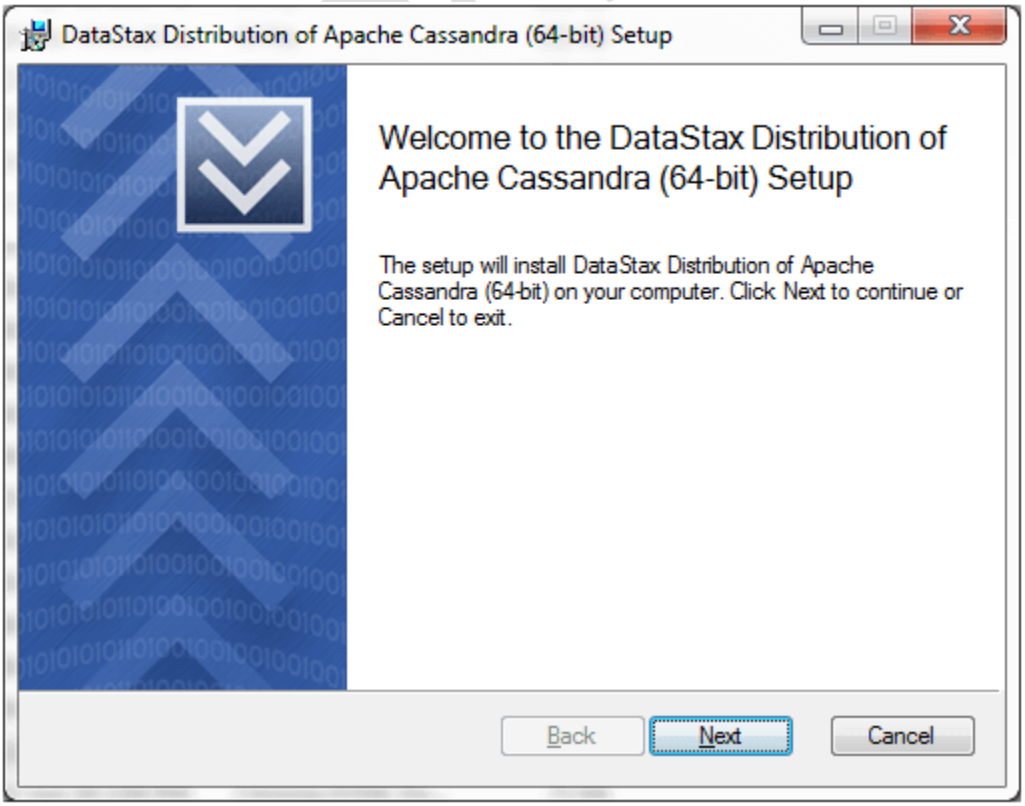
Before installing Apache Cassandra, you must have the following things:

- You must have datastax community edition. You can download Cassandra click here
- JDK must be installed.
- Platform should be Window.

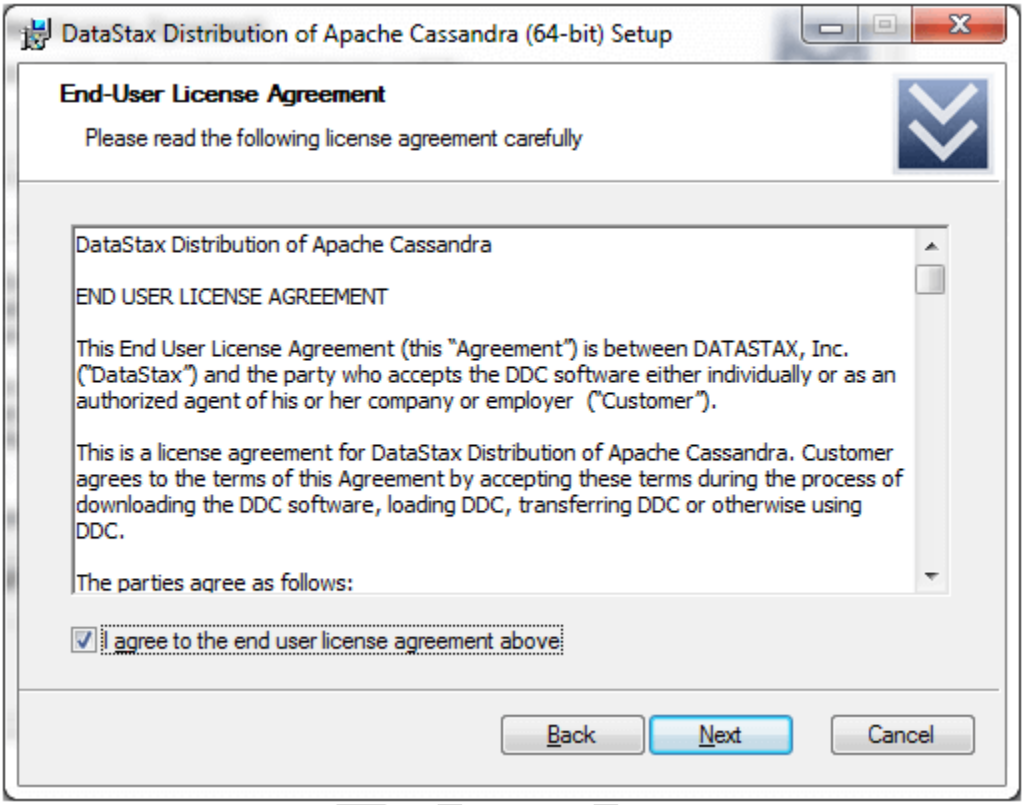
Download and Install Cassandra

Follow the steps given below:

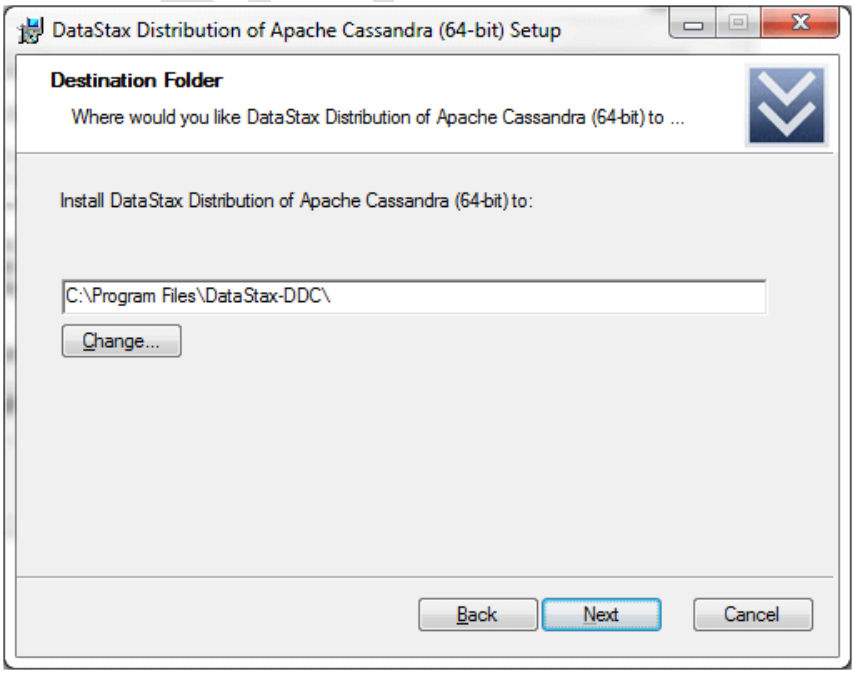
- Run the datastax community edition setup. After running the setup, you will see the following page will be displayed. It is a screenshot of 64 bit version.



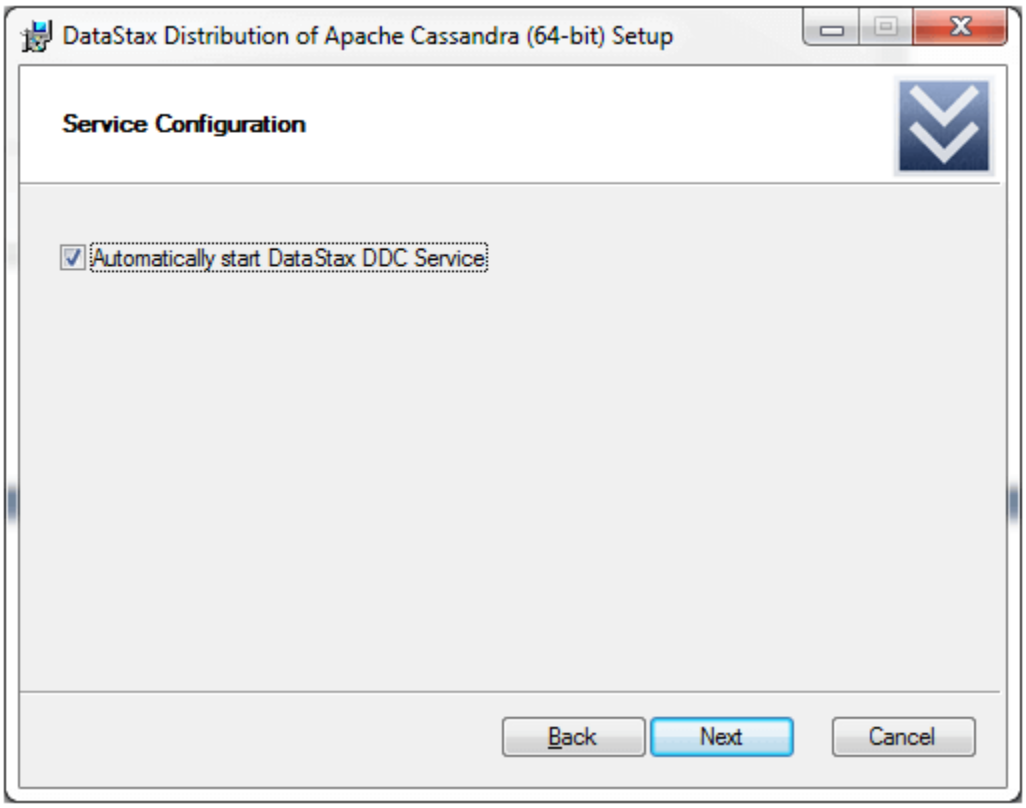
- Click on the next button and you will get the following page:

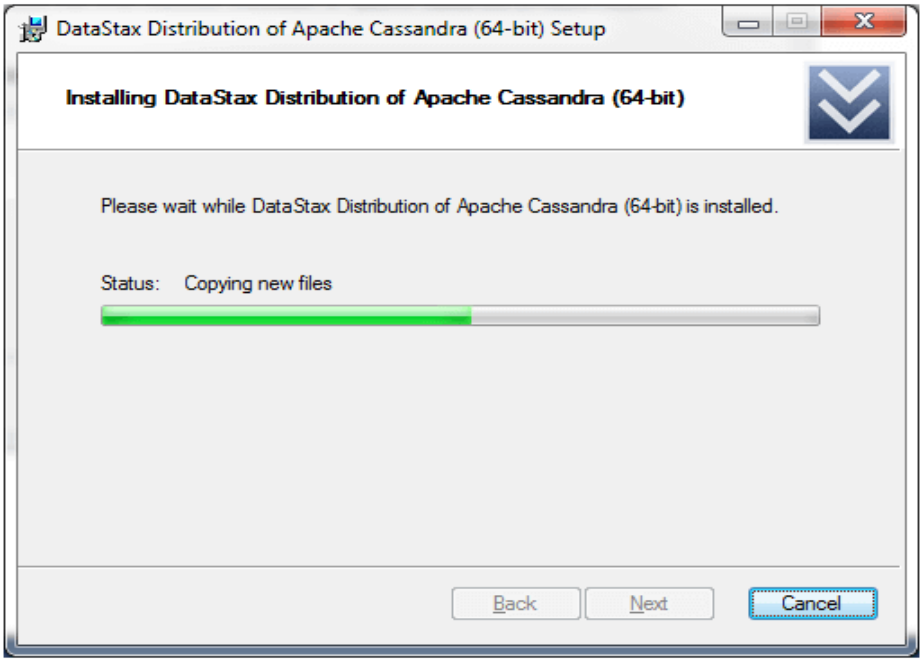


- Press the next button and you will get the following page specifying the location of the installation.

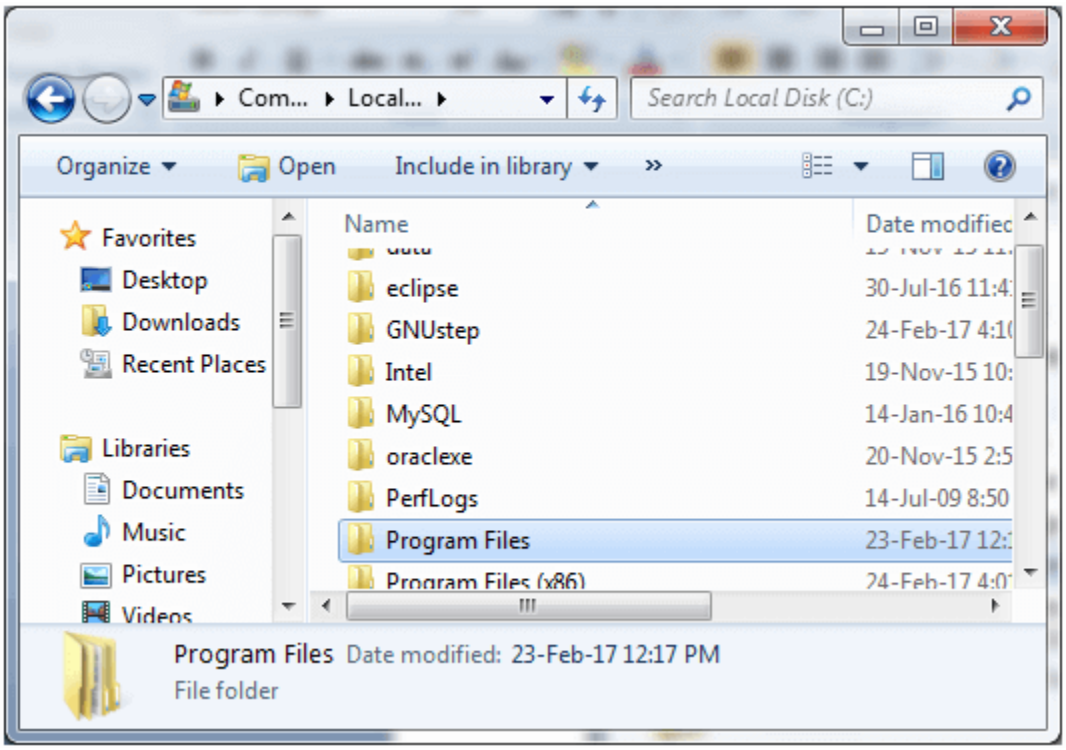


- Press the next button and a page will appear asking about whether you automatically start Data Stax DDC service. Click on the radio button and proceed next.

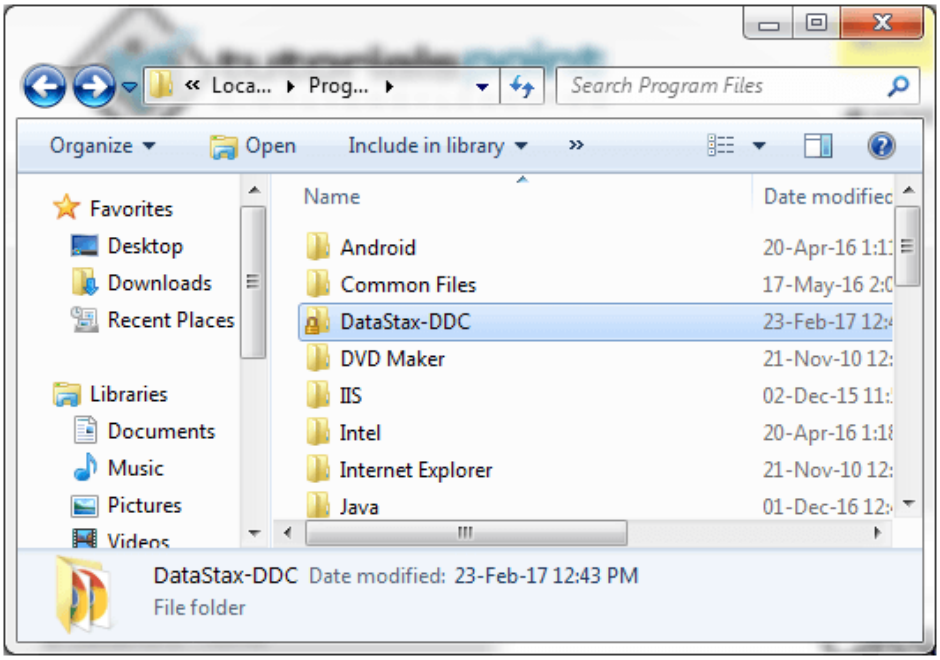




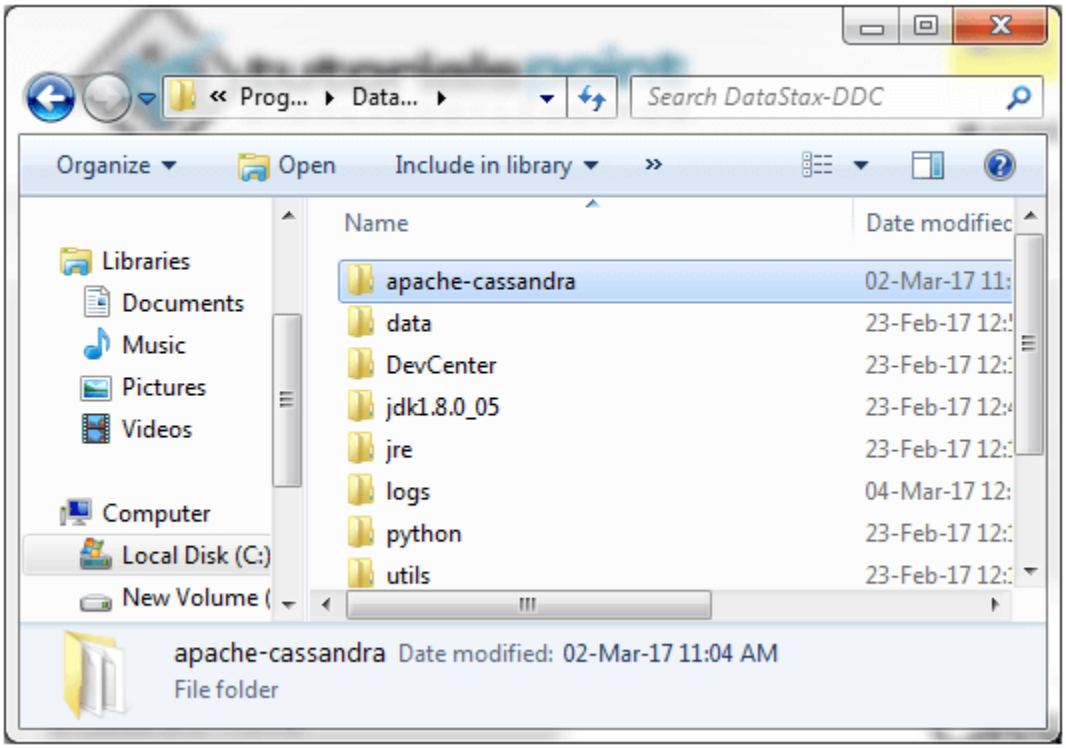
- Installation is started now. After completing the installation, go to program files where Data Stax is installed.



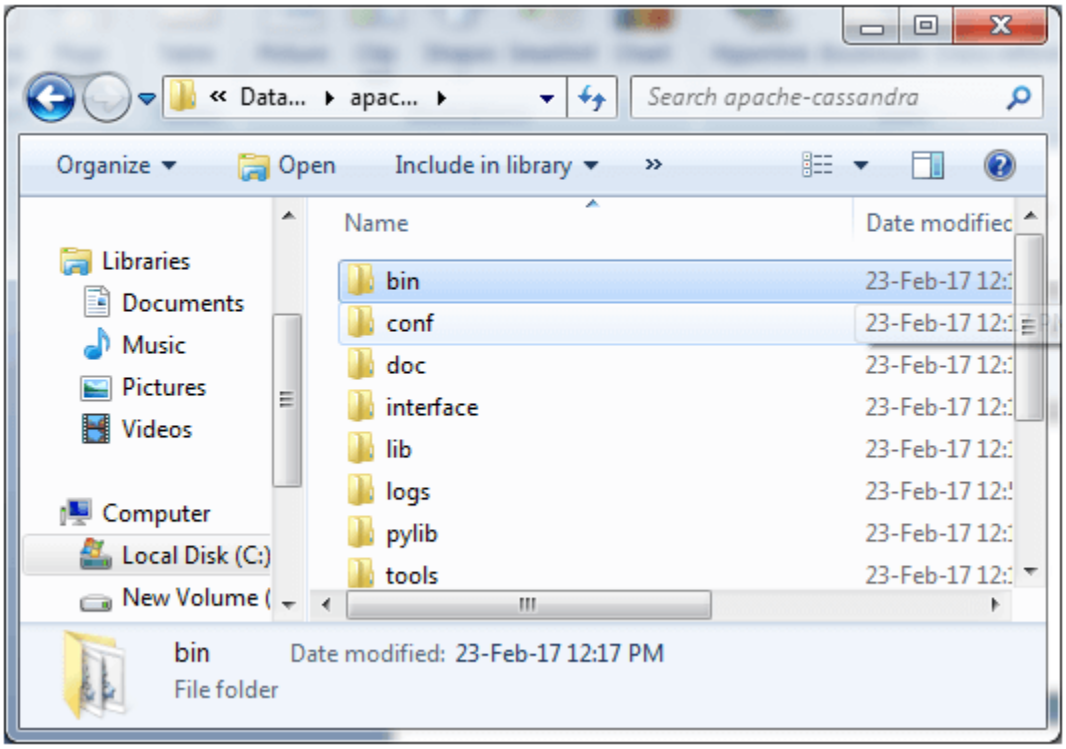
- Open Program Files then you see the following page:



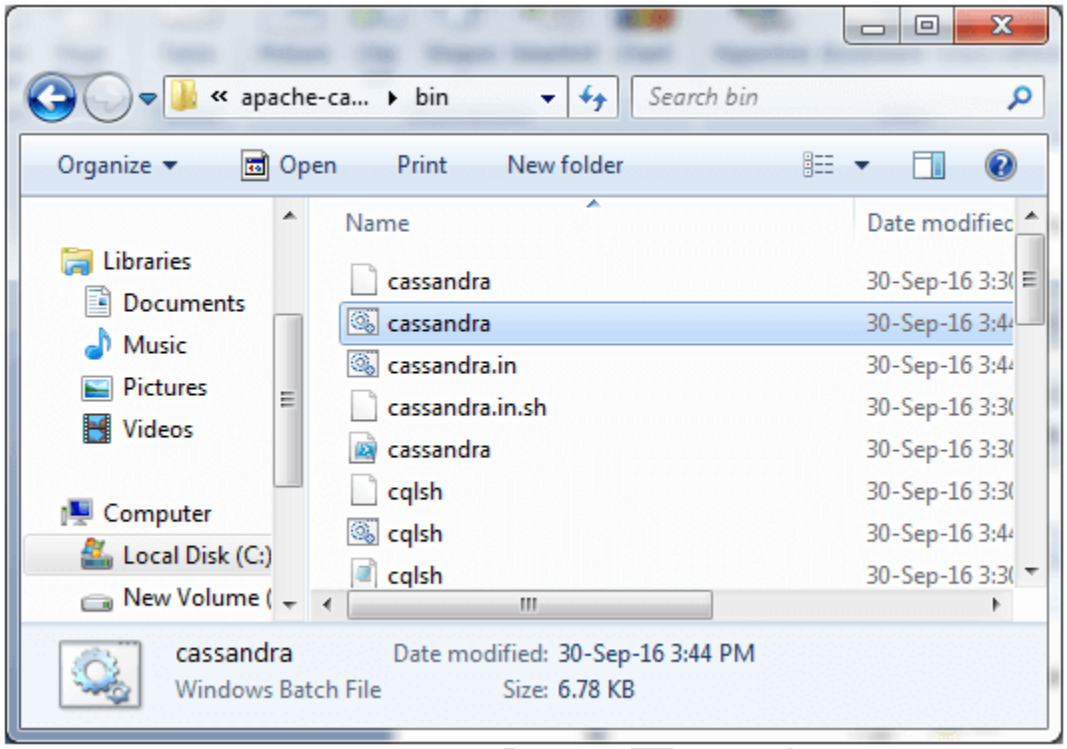
- Open DataStax-DDC then you see Apache Cassandra:



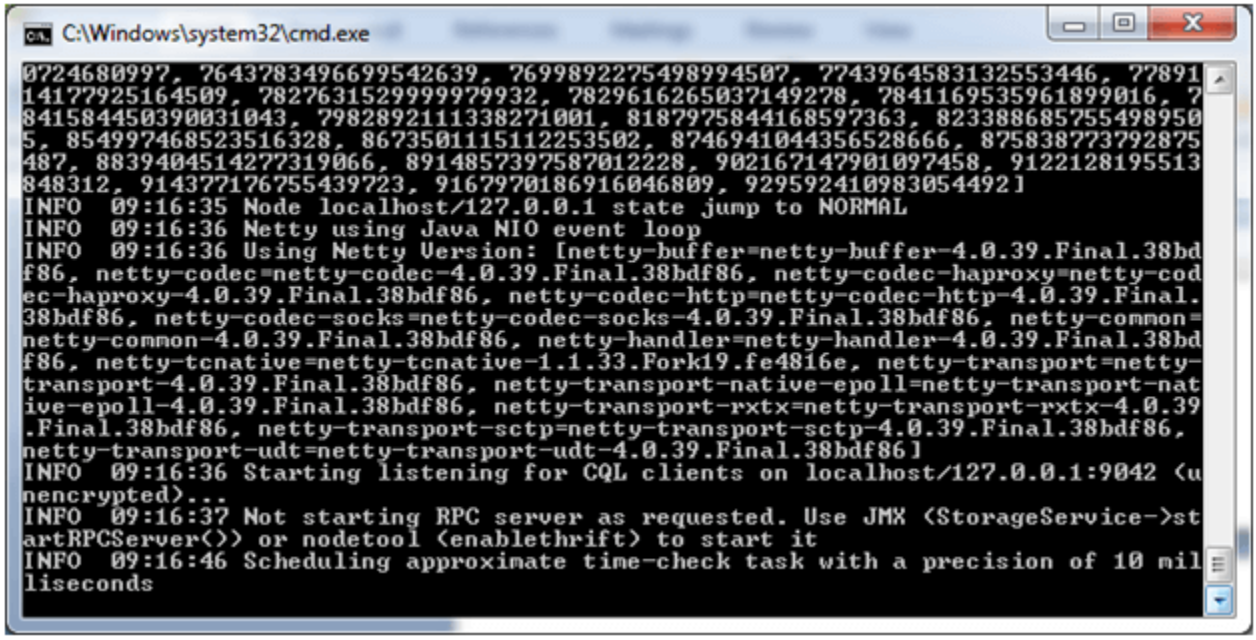
- Open Apache Cassandra and you see bin:



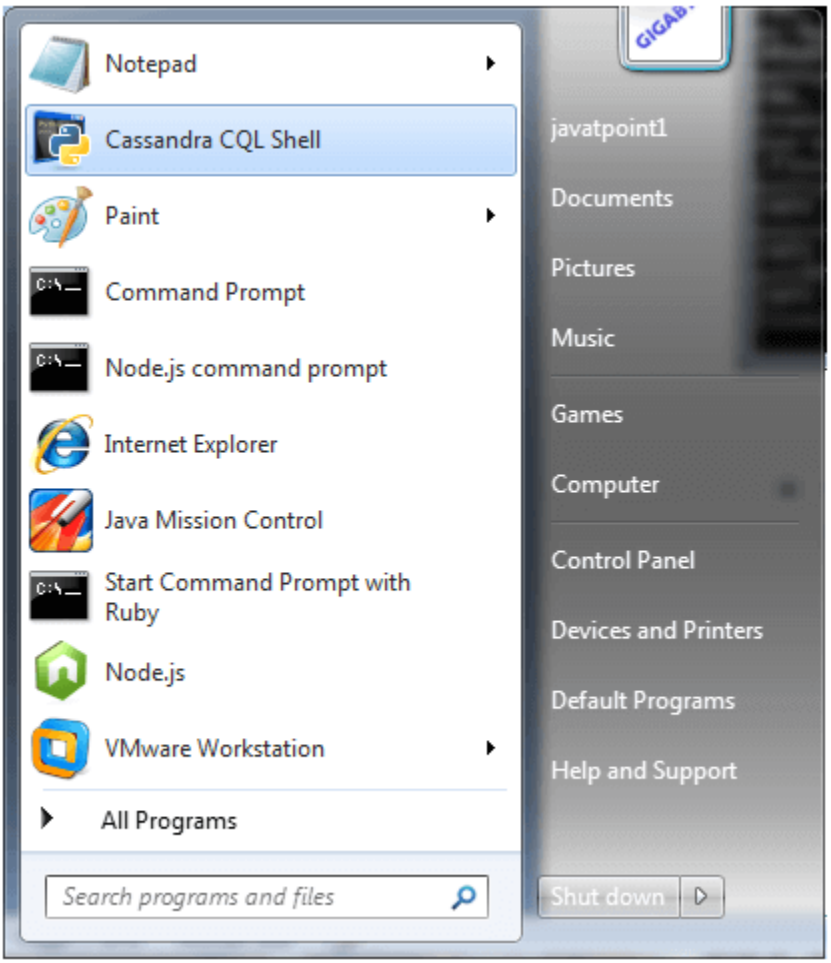
- Open bin and you will see Cassandra Windows batch File:



- Run this file. It will start the Cassandra server and you will see the following page:



- Server is started now go to windows start programs, search Cassandra CQL Shell



❖ 3.5 Basic operations and maintenance

Creating Tables

The first operation in Cassandra is to create a table. A table is defined by set of columns. Each column has name, data type, and an optional value. To create a table, you need to specify the keyspace. It is namespace that defines the replication strategy, and the table name. we also need to specify the columns and their data types.

Example-1

```
CREATE TABLE example_table (
  id int PRIMARY KEY,
  name text,
  age int
```

);

Inserting Data

After defining and creating a dabase table. You can insert data into it. To insert data, You need to specify keyspace, table name, and values for each column. You can use the NULL keyword when a column has no value.

Example-1

INSERT INTO example_table (id, name, age) VALUES (1, 'Yash', 30);

Updating Data

To update data, you can use the UPDATE statement. You need to specify keyspace, table name, columns to update, and new values. You also need to specify the WHERE clause to identify the row to update.

Example-1

UPDATE example_table SET age = 30 WHERE id = 1;

Deleting Data

To delete data, you can use the DELETE statement. you need to specify keyspace, table name, and row to delete. You also need to specify the WHERE clause to identify the row to delete.

Example-1

DELETE FROM example_table WHERE id = 1;

Selecting Data

You can use the SELECT statement to select data. You need to specify keyspace, table name, and columns to retrieve. You also need to specify the WHERE clause to filter the rows. You can use the ORDER BY clause to sort the rows. The LIMIT clause to limit the number of rows returned.

Example-1

SELECT name, age FROM example_table WHERE age > 30 ORDER BY age DESC LIMIT 10;

ALTER TABLE

To modify an existing table structure in Cassandra, you can use the ALTER TABLE statement.

Example

ALTER TABLE users
ADD surname TEXT;

It adds a new column named 'surname' of type 'TEXT' to the 'users' table.

DROP TABLE

To delete an existing table in Cassandra. You can use the DROP TABLE statement.
Example
DROP TABLE users;

❖ 3.6 Monitoring and troubleshooting

Monitoring using the nodetool utility:

- The nodetool utility is a command-line interface for monitoring Cassandra and performing routine database operations.
- Included in the Cassandra distribution, nodetool and is typically run directly from an operational Cassandra node.
- The nodetool utility supports the most important JMX metrics and operations, and includes other useful commands for Cassandra administration, such as the proxyhistogram command.

Example:
Status and Information:

Node Status:

```
nodetool status
```

Displays the status of each node in the cluster, including their state (UN - Up, DN - Down), load, and tokens.

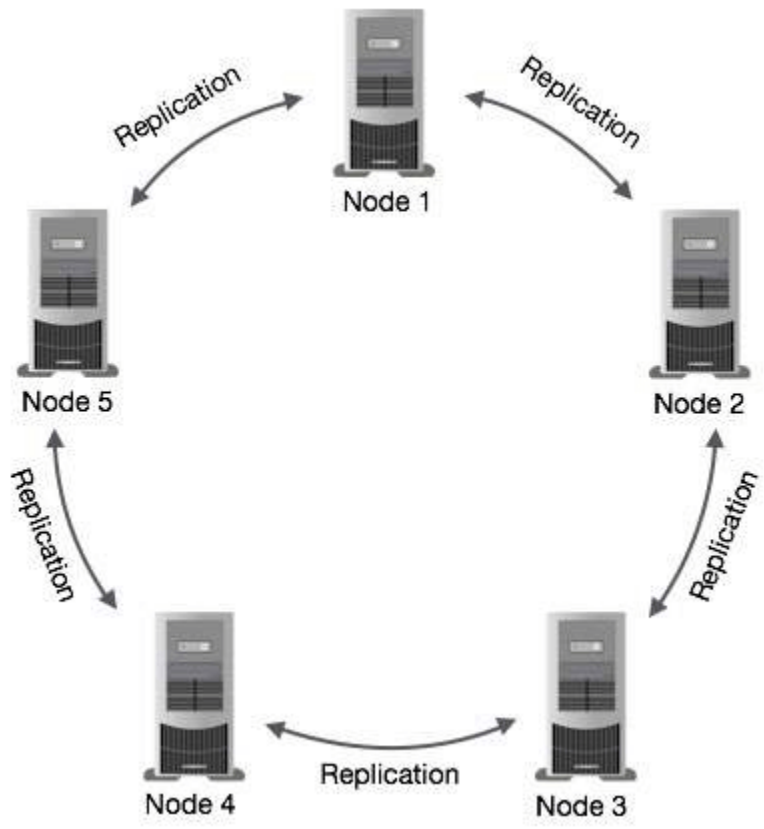
Cluster Information:

```
nodetool info
```

❖ 3.7 Cassandra architecture

- It has a peer-to-peer distributed system across its nodes, and data is distributed among all the nodes in a cluster.
- In Cassandra, each node is independent and at the same time interconnected to other nodes. All the nodes in a cluster play the same role.
- Every node in a cluster can accept read and write requests, regardless of where the data is actually located in the cluster.

- In the case of failure of one node, Read/Write requests can be served from other nodes in the network.
- In Cassandra, nodes in a cluster act as replicas for a given piece of data. If some of the nodes are responded with an out-of-date value, Cassandra will return the most recent value to the client. After returning the most recent value, Cassandra performs a read repair in the background to update the stale values.



Components of Cassandra

The main components of Cassandra are:

- **Node:** A Cassandra node is a place where data is stored.
- **Data center:** Data center is a collection of related nodes.
- **Cluster:** A cluster is a component which contains one or more data centers.
- **Commit log:** In Cassandra, the commit log is a crash-recovery mechanism. Every write operation is written to the commit log.

- **Mem-table:** A mem-table is a memory-resident data structure. After commit log, the data will be written to the mem-table. Sometimes, for a single-column family, there will be multiple mem-tables.
- **SSTable:** It is a disk file to which the data is flushed from the mem-table when its contents reach a threshold value.
- **Bloom filter:** These are nothing but quick, nondeterministic, algorithms for testing whether an element is a member of a set. It is a special kind of cache. Bloom filters are accessed after every query.

Cassandra Query Language

Cassandra Query Language (CQL) is used to access Cassandra through its nodes. CQL treats the database (Keyspace) as a container of tables. Programmers use cqlsh: a prompt to work with CQL or separate application language drivers.

The client can approach any of the nodes for their read-write operations. That node (coordinator) plays a proxy between the client and the nodes holding the data.

❖ 3.8 Performance tuning and optimization

One of the most important factors in improving Cassandra read performance is to optimize data modeling.
The key to this optimization is to design your data model around your queries.
This means that you should think about how your data will be accessed and structured your data model accordingly.

Tune compaction

Avoid reads on SSTables, and thus on disks cannot be always avoided. In this case, we want to limit the amount of I/Os to the maximum.
Our problem is that writes to given key might be spread across several SSTables. For instance, this is the case for tables with many updates, deletes or with Clustering keys.
We can hopefully trade compaction time and I/Os against read efficiency.

❖ 3.9 Compaction strategies

Size Tiered Compaction Strategy to **Levelled Compaction Strategy**
Levelled compaction strategy will by its structure limit the number of SSTables a given key can belong to. Read intensive tables, with updates, deletes or partition keys will benefit a lot from this change. Avoid it on immutable, not clustered tables, as you will not get collocation benefits, but will pay the costs of more expensive compactions.

Date Tiered Compaction Strategy stores data written within a certain period of time in the same SSTable. It's very useful for time series of tables implying the use of TTL, where entries expire.

Updating the compaction strategy of your tables can be done without downtime, at the cost of running compactions. Warning: this might consume IO and memories, and thus decrease performances when the compaction is running.
You need to modify the CQL table declaration to change the compaction strategy:

```
use apache_james ;
ALTER TABLE modseq
WITH compaction = { 'class' : 'LeveledCompactionStrategy' };
```

For the changes to take effect, you need to compact the SSTables. To force this, you need to use nodetool:
nodetool compact keyspace table

VPMP POLYTECHNIC